

Seminar Project Progress Report – II

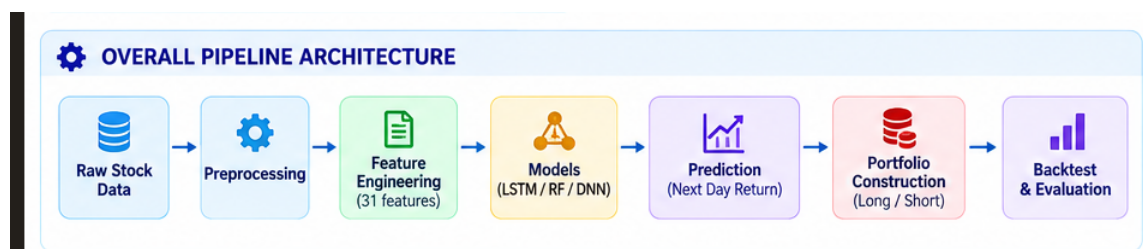
Implement baseline model

Roll Number: MA25C033

Name: Rahul

This Report Target

1. Implement Fischer & Krauss (2018), "Deep learning with LSTM networks for financial market predictions"



1 Implementation Progress

1.1 Environment Setup

To ensure reproducibility of the implementation, a dedicated Conda environment named `stock_lstm` was created. The implementation is being developed using Python 3.11 and the following core libraries:

Table 1: Software Environment

Package	Version
Python	3.11
yfinance	1.7.0
pandas	2.2.3
NumPy	2.4.6
Matplotlib	Installed
Scikit-Learn	Installed
SciPy	Installed
Jupyter Notebook	Installed
Requests	Installed

The objective of Notebook–1 is to acquire, organize, and validate the financial dataset required for reproducing the methodology proposed by Fischer and Krauss (2018).

1.2 Notebook–1: Data Collection and Preparation

The first notebook focuses exclusively on data acquisition, storage, and quality assessment. At this stage, no machine learning model has been trained.

1.2.1 Company Metadata Collection

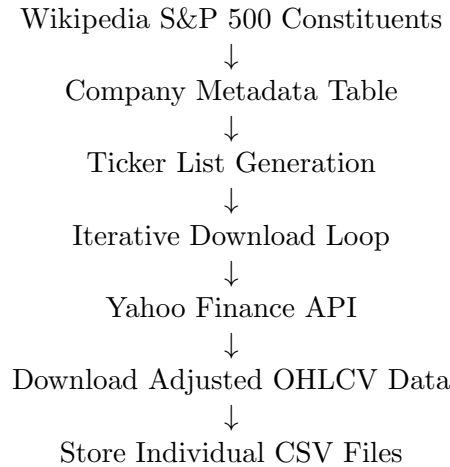
Company metadata were obtained from the current S&P 500 constituent table available on Wikipedia. The downloaded information includes:

- Ticker Symbol
- Company Name
- GICS Sector
- GICS Sub-Industry

Unlike the original study, historical index membership and delisted companies are not available through this source. Consequently, the reproduced dataset is subject to survivorship bias, whereas the original paper used a professional financial database containing historical constituents.

1.2.2 Data Acquisition Pipeline

The complete data acquisition workflow implemented in Notebook–1 is illustrated below:



1.2.3 Notebook Structure

The notebook is organized into the following stages:

- (a) Import required libraries.
- (b) Define directory paths and configuration parameters.
- (c) Retrieve the S&P 500 constituent table from Wikipedia.
- (d) Generate the ticker universe.
- (e) Download historical stock data from Yahoo Finance.
- (f) Save individual stock datasets.
- (g) Download benchmark index data (̂SPC).

- (h) Perform coverage diagnostics.
- (i) Construct the price panel and corporate actions database.
- (j) Visual inspection of selected securities.

1.2.4 Downloaded Financial Variables

For each successfully downloaded stock, the following variables are collected:

- Date
- Open Price
- High Price
- Low Price
- Close Price (Adjusted)
- Trading Volume
- Dividends
- Stock Splits

The adjusted closing price provided by Yahoo Finance is used as a practical proxy for the total-return series required in the original study.

1.2.5 Coverage Diagnostics

Coverage diagnostics were computed for all downloaded securities. For each stock, the following information was evaluated:

- Number of observations
- First available trading date
- Last available trading date
- Coverage ratio
- Maximum gap between observations

Low coverage ratios may arise either from missing observations or from firms entering the market after the beginning of the study period. Therefore, coverage statistics are used as a diagnostic measure rather than an automatic indicator of poor data quality.

1.2.6 Price Panel Construction

All individual stock files were consolidated into a single panel dataset indexed by trading date.

$$\text{Price Panel} = \begin{bmatrix} P_{1,1} & P_{1,2} & \cdots & P_{1,N} \\ P_{2,1} & P_{2,2} & \cdots & P_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ P_{T,1} & P_{T,2} & \cdots & P_{T,N} \end{bmatrix} \quad (1)$$

where rows represent trading dates and columns represent stocks.

This step consolidates all individual stock price histories into a single panel dataset while separately recording dividend and stock-split events for transparency and future data-quality analysis.

1.2.7 Benchmark Index Data

The S&P 500 benchmark index ($\hat{GSPC}$) was downloaded separately and stored for later comparison against portfolio performance.

1.2.8 Visual Data Inspection

As a final validation step, a small sample of stocks was plotted using adjusted closing prices normalized to 100 at the first available observation.

This visual inspection serves as a sanity check and helps identify:

- Missing observations
- Abnormal price jumps
- Incorrect download histories
- Data corruption issues

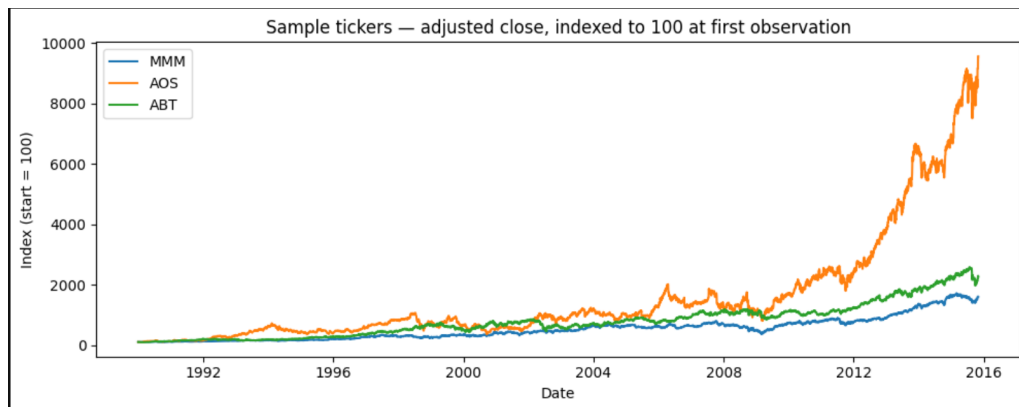


Figure 1: Sample adjusted stock prices normalized to 100 at the first available observation.

Table 2: Comparison of Original Study Environment and Current Implementation

Component	Original Paper (Fischer & Krauss, 2018)	Current Implementation
Programming Language	Python 3.5	Python 3.11
Data Handling	NumPy, Pandas	NumPy 2.4.6, Pandas 2.2.3
Data Source	Thomson Reuters Datastream	Yahoo Finance (yffinance 1.7.0)
LSTM Framework	Keras on top of TensorFlow	Planned: TensorFlow/Keras (to be implemented)
Random Forest	scikit-learn	scikit-learn
Logistic Regression	scikit-learn	scikit-learn
Deep Neural Network	H2O Framework	Planned: TensorFlow/Keras implementation
Performance Evaluation	R + PerformanceAnalytics	Python (NumPy, Pandas, Matplotlib, SciPy)
Visualization	Not explicitly stated	Matplotlib
Data Download	Institutional Financial Database	Yahoo Finance API
Hardware	NVIDIA GPU (LSTM), CPU Cluster (Other Models)	Personal Computer (CPU), GPU usage NVIDIA RTX 2050

2 Notebook 2: Data Preparation and Study Period Construction

The objective of Notebook 2 is to transform the raw price data collected in Notebook 1 into the exact training, validation, and testing datasets required by the forecasting framework. This includes the construction of binary classification targets, generation of rolling study periods, creation of 240-day LSTM sequences, extraction of benchmark features, and verification of data integrity.

2.1 Price Panel Loading

The consolidated price panel generated in Notebook 1 was loaded for further processing.

- Trading dates: 6511
- Stocks: 609
- Date range: 02-Jan-1990 to 30-Oct-2015

Table 3: Price Panel Summary

Metric	Value
Trading Dates	6511
Stocks	609
Start Date	1990-01-02
End Date	2015-10-30

2.2 Target Variable Construction

Following the methodology of Fischer and Krauss (2018), daily stock returns were computed and transformed into a binary classification target.

For each trading day, the cross-sectional median return across all stocks was calculated. A stock receives:

$$Y_{i,t+1} = \begin{cases} 1, & \text{if } r_{i,t+1} \geq \text{Median}(r_{t+1}) \\ 0, & \text{otherwise} \end{cases}$$

The resulting class distribution is nearly balanced by construction.

Table 4: Target Class Distribution

Class	Proportion
1 (Outperform Median)	0.5085
0 (Underperform Median)	0.4915

2.3 Rolling Study Period Generation

The dataset was partitioned into rolling study periods using the framework proposed in the original paper.

- Training window: 750 trading days
- Trading/Test window: 250 trading days
- Study period length: 1000 trading days
- Window shift: 250 trading days

The first three generated study periods are shown below.

Table 5: First Three Rolling Study Periods

Study	Train Start	Train End	Trade Start	Trade End
1	1990-01-02	1992-12-16	1992-12-17	1993-12-13
2	1990-12-27	1993-12-13	1993-12-14	1994-12-08
3	1991-12-23	1994-12-08	1994-12-09	1995-12-05

2.4 Selected Study Period

For experimentation, the final rolling study period was selected.

Table 6: Working Study Period

Metric	Value
Study Period	23
Training Window	2011-10-25 to 2014-10-17
Training Days	750
Trading/Test Window	2014-10-20 to 2015-10-15
Trading Days	250

2.5 LSTM Dataset Construction

For the LSTM model, each sample consists of a rolling sequence of 240 standardized daily returns.

The training window was further split chronologically into training and validation subsets.

Table 7: LSTM Dataset Statistics

Split	Samples	Shape	Positive Rate
Train	341257	(341257,240,1)	0.501
Validation	87579	(87579,240,1)	0.501
Test	148398	(148398,240,1)	0.501

2.6 Benchmark Dataset Construction

To reproduce the benchmark models used in the paper (Logistic Regression, Random Forest, and DNN), 31 cumulative-return features were generated.

The horizons used were:

$$1, 2, \dots, 20, 40, 60, \dots, 240$$

resulting in 31 engineered features per stock-day observation.

Table 8: Benchmark Dataset Statistics

Split	Rows	Columns
Train	341257	32
Validation	87579	32
Test	148398	32

The 32 columns consist of:

$$31 \text{ engineered features} + 1 \text{ target variable}$$

2.7 Generated Output Files

The processed datasets were stored for subsequent model-training notebooks.

Table 9: Files Generated for Study Period 23

File	Description
lstm_train.npz	LSTM training sequences
lstm_val.npz	LSTM validation sequences
lstm_test.npz	LSTM testing sequences
benchmark_train.csv	Benchmark training data
benchmark_val.csv	Benchmark validation data
benchmark_test.csv	Benchmark testing data

2.8 Sanity Checks

Several validation checks were performed before proceeding to model training:

- No NaN values remained in the final datasets.
- Target distributions remained balanced across all splits.
- Training, validation, and testing dates were mutually exclusive.
- Sequence dimensions matched the required 240-day lookback structure.
- Benchmark feature matrices contained all 31 cumulative-return features.

These checks confirm that the generated datasets are internally consistent and satisfy the requirements of the forecasting framework.

PROJECT DIRECTORY TREE

- lstm-stock-prediction/
 - data/
 - raw/ # per ticker
 - processed/ # features
 - notebooks/ # 8 total
 - 01_data_processing.ipynb
 - 02_feature_engineering.ipynb
 - 03_baseline_logreg.ipynb
 - 04_lstm_model.ipynb
 - 05_random_forest.ipynb
 - 06_deep_neural_net.ipynb
 - 07_backtest_eval.ipynb
 - 08_results_comparison.ipynb
 - src/ # modules
 - data_download.py
 - data_prep.py
 - models.py
 - metrics.py
 - backtest.py
 - results/
 - metrics/
 - plots/
 - models/
 - docs/
 - implementation_plan.docx
 - report.docx
 - requirements.txt
 - README.md

NOTEBOOKS — WEEK BY WEEK ROADMAP

Week 1 Data & Features	01	Data processing	Multi-source download (Yahoo, Stooq, AlphaVantage), S&P 500 tickers, survivorship-bias note	Output • Raw price data for all tickers • Data statistics and visualization
	02	Feature engineering	Returns, median-split target, 240-day sequences, chronological split	Output • Engineered features (31 features) • Train/Val/Test splits
Week 2 Baseline & LSTM	03	Baseline logistic regression	31 cumulative-return features, pipeline verification before LSTM	Output • Baseline performance • Verification of pipeline
	04	LSTM model	240-day sequences, 25 hidden units, dropout 0.1, RMSprop, early stopping	Output • Trained LSTM model • Training curves and validation results
Week 3 Benchmark Models	05	Random forest	1000 trees, max depth 20, same 31 features	Output • Trained RF model • Feature importance
	06	Deep neural net	31 → 31 → 10 → 5 → 2, maxout activation, dropout 0.5	Output • Trained DNN model • Training and validation results
Week 4 Evaluation & Report	07	Backtest & evaluation	Long-short portfolio, Sharpe/Sortino/VaR, DM & PT tests	Output • Backtest results • Risk-adjusted performance metrics
	08	Results comparison	Vs. paper's published numbers, black-box pattern analysis	Output • Comparison table and plots • Final report and conclusions

3 Notebook 3: Rolling Logistic Regression and Portfolio Backtesting

3.1 Objective

The objective of Notebook 3 is to replicate the benchmark machine learning framework presented in the paper for stock return prediction. A Logistic Regression classifier is trained using historical return-based features and is evaluated through a realistic rolling-window backtesting framework.

Unlike a traditional classification task, the final objective is not merely prediction accuracy but the generation of economically meaningful alpha through long-short portfolio construction.

3.2 Data Overview

The study uses the S&P 500 universe from January 1990 to October 2015.

- Number of trading days: 6511
- Number of stocks: 609
- Prediction universe: 599–609 stocks depending on availability
- Final test observations: 2,683,205 stock-day pairs
- Combined test period: 5750 trading days

The binary target variable indicates whether a stock outperforms the cross-sectional median return on the next trading day.

3.3 Features

The model uses lagged return features at multiple horizons:

- 1-day return
- 2-day return
- 3-day return
- 5-day return
- 6-day return
- 12-day return
- 13-day return
- 19-day return
- 20-day return
- 60-day return

All features are standardized using the training set statistics before model fitting.

3.4 Rolling Window Framework

Following the methodology described in the paper, a rolling-window evaluation procedure is used.

For each study period:

- (a) Train model using historical data.
- (b) Select hyperparameters using a validation set.
- (c) Generate out-of-sample predictions on the trading period.
- (d) Roll the window forward.

A total of 23 rolling study periods are evaluated.

This procedure avoids look-ahead bias and closely resembles a real-world trading environment.

3.5 Model Training

A Logistic Regression classifier with L2 regularization is employed.

The regularization parameter C is selected using cross-validation.

For every rolling period:

$$P(y = 1|X) = \frac{1}{1 + \exp(-(\beta_0 + \beta^T X))} \quad (2)$$

where

- X denotes the standardized return features
- y denotes the next-day directional target

4 Comparison with the Original Paper

Component	Paper	Our Implementation
Universe	S&P500	S&P500
Period	1990–2015	1990–2015
Rolling windows	Yes	Yes
Logistic Regression	Yes	Yes
Cross-validation	Yes	Yes
Top/Bottom portfolio construction	Yes	Yes
Transaction costs	Yes	Yes
Sector analysis	Limited	Yes
Random monkey benchmark	Yes	Yes
Quarterly stability analysis	No	Yes
Turnover-based cost model	No	Yes
Reversal benchmark	No	Yes

Table 10: Comparison between the paper and our implementation

5 Classification Performance

The pooled classification accuracies across all rolling periods are

Dataset	Accuracy
Training	51.55%
Validation	51.24%
Testing	51.06%

Although the accuracy improvement above random guessing appears small, financial prediction problems are extremely noisy and even a small directional edge may translate into substantial trading profits.

6 Feature Importance

The average coefficients across the 23 rolling periods reveal that the most influential features are short-term return variables.

Feature	Mean Coefficient
ret_1d	-0.0236
ret_2d	-0.0228
ret_6d	-0.0198
ret_3d	-0.0102
ret_19d	-0.0099

The predominance of negative coefficients suggests that the model primarily exploits short-term mean-reversion effects rather than momentum effects.

7 Portfolio Construction

For each trading day:

- (a) Stocks are ranked by predicted probability.
- (b) Top- K stocks are placed in the long portfolio.
- (c) Bottom- K stocks are placed in the short portfolio.
- (d) Equal weighting is used within each leg.

The benchmark setting uses

$$K = 10$$

leading to a daily long-short portfolio consisting of 20 stocks.

8 Economic Performance

The long-short strategy generates positive returns throughout the testing period.

Metric	Gross	Net
Mean Daily Return	0.92%	0.72%
Annual Return	694.9%	381.8%
Sharpe Ratio	8.48	4.66
Trading Days	5750	5750

Table 11: Portfolio performance

9 Statistical Significance Tests

To verify that the observed performance is not due to random chance, two statistical tests are conducted.

9.1 Binomial Test

The top-flop portfolio achieves

$$53.5\%$$

directional accuracy.

The corresponding binomial test produces

$$p < 0.001$$

indicating statistically significant predictive power.

9.2 Pesaran-Timmermann Test

The Pesaran-Timmermann statistic is

$$PT = 23.715$$

with

$$p < 0.001$$

which confirms significant market timing ability.